

Road Lane Detection and Analysis - Project Report

1. Cover Page

Project Title: Road Lane Detection and Analysis Using Classical Computer Vision

Course: VITYarthi Build Your Own Project

Student Name: Submitted for Review

2. Introduction

This project implements a command-line computer vision system to detect left and right road lane boundaries from images using purely classical image processing techniques without any machine learning or deep learning.

3. Problem Statement

Understanding the road layout is a fundamental task for autonomous driving. While modern systems use deep learning, it is crucial to understand classical geometric approaches to line detection. This project builds an understandable, CPU-friendly pipeline to extract lane lines and evaluates it against a synthetically generated ground-truth dataset.

4. Functional Requirements

- Generate synthetic road image dataset with known ground truth.
- Detect edges and lines using classical CV techniques.
- Extract, separate, and fit left and right lane boundaries.
- Provide a CLI to run single-image and batch detections.
- Automatically evaluate accuracy using Mean Absolute Error (MAE).

5. Non-functional Requirements

- Performance: Must run efficiently on a CPU using OpenCV.
- Maintainability: Code must be modularized into discrete steps (preprocessing, edge detection, etc.).
- Usability: Simple argparse CLI for executing all workflows.
- Reliability: Graceful error handling for missing files and empty datasets.

6. System Architecture

The system uses a sequential pipeline: Grayscale -> Gaussian Blur -> Canny Edge Detection -> ROI Masking -> Hough Transform -> Lane Separation -> Output Annotation.

7. Design Diagrams

The system architecture and workflow are documented in Mermaid diagrams located in the 'docs/' directory. They outline the sequential pipeline and user interaction patterns via the CLI.

8. Design Decisions & Rationale

A classical CV pipeline was chosen instead of ML to emphasize fundamental image processing concepts. A synthetic dataset was preferred so that exact ground-truth pixel coordinates could be used to calculate a precise MAE metric rather than relying on manual labeling.

9. Implementation Details

The image is converted to grayscale before edge detection. The Canny operator is then applied to identify strong edges. A trapezoidal region is used as the region of interest to filter out background scenery. Detected Hough lines are divided into left and right groups using their slopes (positive/negative) and screen positions, and a degree 1 polynomial is fit to approximate the continuous lane lines.

10. Screenshots / Results

Actual evaluation on 90 test images yielded the following results:

- Left Lane Detection Rate: 0.87
- Right Lane Detection Rate: 0.71
- Left Lane MAE: 14.20 pixels
- Right Lane MAE: 17.18 pixels

Output images with overlaid lane lines were successfully saved to the outputs/ directory.

11. Testing Approach

Automated testing was conducted using Pytest. Tests verify that grayscale outputs are 2D, ROI masks properly occlude regions, Hough lines are successfully separated based on geometry, and the pipeline executes without crashing on valid inputs.

12. Challenges Faced

Tuning Canny thresholds and Hough Line minimum lengths was challenging because noisy images caused spurious lines. Incorrect Hough lines sometimes confused the left/right lane separation, requiring stricter slope filtering (ignoring horizontal and near-vertical slopes).

13. Learnings & Key Takeaways

I learned how sensitive classical CV pipelines are to parameter changes and environmental noise. Separating geometric filtering (ROI masking) from pixel-intensity filtering (Canny) is critical for robust detection.

14. Future Enhancements

Future enhancements could include implementing temporal smoothing to track lanes across multiple video frames, applying a perspective transformation to view lanes from a 'bird's eye' view, and utilizing curve fitting

Road Lane Detection and Analysis - Project Report

to detect non-straight lanes.

15. References

- OpenCV Documentation (<https://docs.opencv.org/>)
- NumPy Documentation (<https://numpy.org/doc/>)